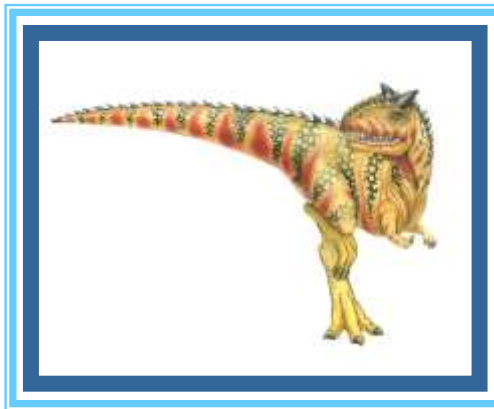
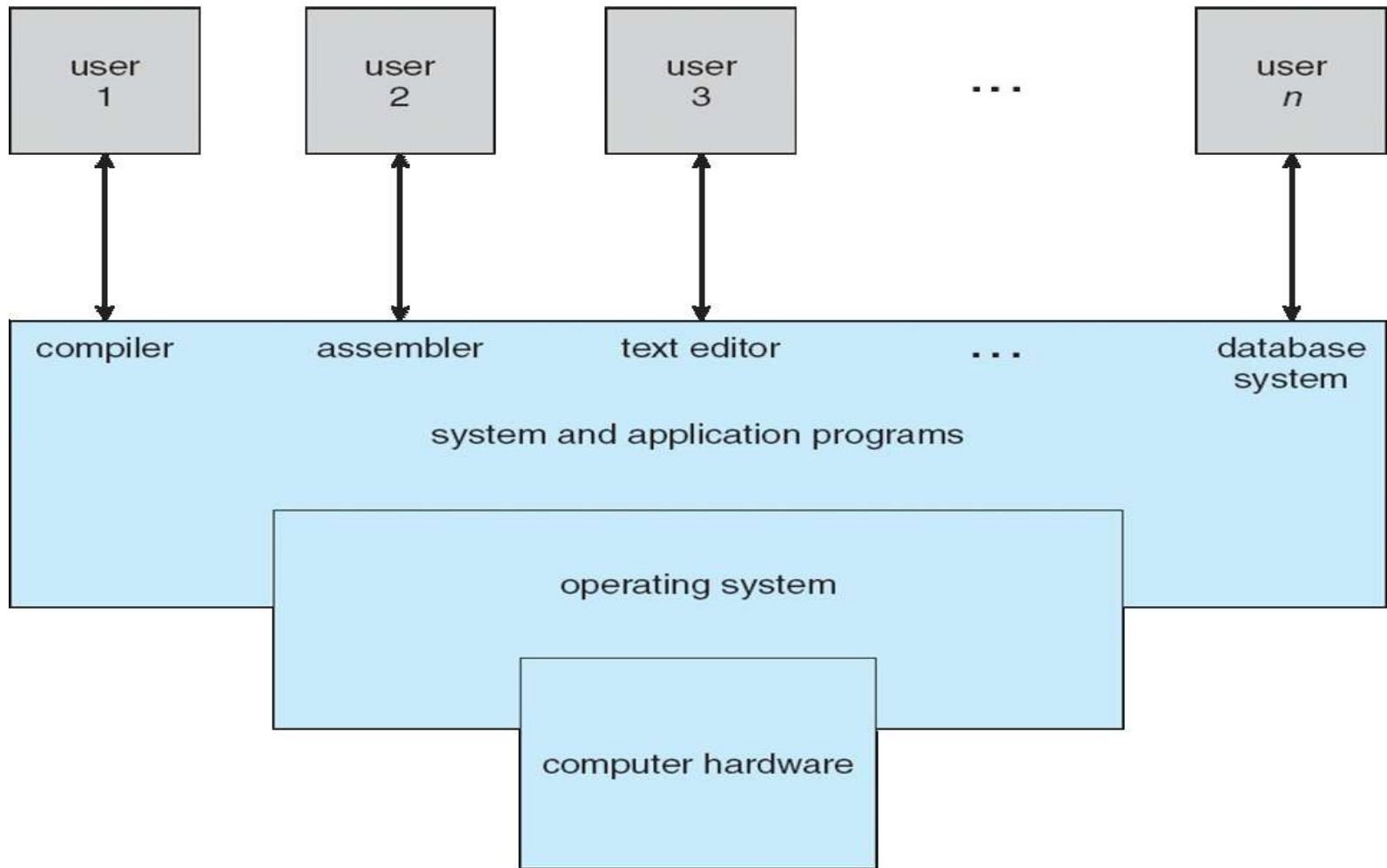


Introduction





Four Components of a Computer System





Four Components of a Computer System

- *Computer system can be divided into **four components**:*
 - **Hardware:** *Provides basic computing resources for the system: CPU, memory, I/O devices*
 - **Operating system:** *Controls and coordinates use of hardware among various applications and users*
 - **Application programs:** *Define the ways in which the system resources are used to solve the computing problems of the users*
 - ▶ *Word processors, compilers, web browsers, database systems, video games*
 - **Users:** *People, machines, other computers*





System and Application Software

■ *System software:*

- *Collection of programs that manage and control the computer hardware,*
- *facilitate communication between hardware components*
- *Provide a platform for running application software.*
- *It acts as an **intermediary** between **user** and **hardware**.*

■ *Application software:*

- *Meant to enable the user to carry out some specific set of tasks or functions.*

e.g. web browser, Microsoft excel, Microsoft word.





What is an Operating System?

Operating system is a *system software*

- OS is an *intermediary* between the *hardware and user-level software*.
- OS makes it possible for applications to interact with the computer's hardware components *without needing* to know the specific details of the underlying hardware.





Operating System's Goals

Convenience + Efficiency

- *Make the computer system convenient to use.*
- *Ensures use of the computer hardware in an efficient manner.*
- *Some operating system are designed to be **convenient** (personal computer and android),*
- *Others to be **efficient** (mainframe, supercomputer)*
- *Some are **combination of both.***





Operating System

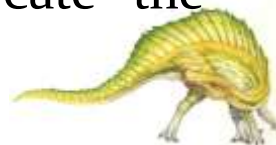
- OS is a **resource allocator/resource manager**.
 - **Manages all resources**, in unbiased fashion so that it can operate the computer system fairly and efficiently.
 - Decides between numerous and conflicting requests for efficient and fair resource use.
- OS is a **control program**.
 - Manages the execution of user programs to prevent errors and improper use of the computer.





Functionalities of Operating System

- **Abstraction:** Hiding hardware's complex details from application developers.
- **Resource Manager:** OS manages all the resources those are attached to the system like memory, processor and all the I/O devices.
- **Process Management :** The processes those are given by the user or the process those are System 's own Process are Handled by the OS.
- **Memory Management:**
 - OS also manages the memory of the computer system
 - It provides the memory to the process and also deallocate the Memory from the process (if a process gets completed).





Functionalities of Operating System

- **Processor Management:** In multiprogramming environment, the OS decides which process gets the processor when and for how much time. This function is called *process scheduling*.
- **Storage Management:** OS also controls all the *storage operations*.
- **Storage operations:** how the data or files will be stored into the computers and how the files will be accessed by the users etc
- **Device Management:** An OS manages device communication via their respective drivers. It keeps tracks of all devices.
 - Decides which process gets the device when and for how much time.
 - Allocates the device in the efficient way.
 - De-allocates devices.





Functionalities of Operating System

- **Security:** The OS uses password protection to protect user data and similar other techniques. it also prevents unauthorized access to programs and user data.
- **Job accounting:** It is function available on every i5/OS™ system that can be used to track usage of system resources.
- **Error detecting aids:** The OS constantly needs to be aware of possible errors.

Errors may occur in:

- CPU and memory hardware (such as a memory error or a power failure).
- I/O devices (a connection failure on a network, or lack of paper in the printer).
- The user program (such as an arithmetic overflow, an attempt to access an illegal memory location, or a too-great use of CPU time).

For each type of error, the operating system should take the appropriate action to ensure correct and consistent computing.





Types of OS

RAM

Uni-programming OS → OS allows only one process to reside in main memory

OS

P

Multiprogramming OS → (Single process cannot keep CPU and I/O devices busy simultaneously)

OS allows multiple processes to reside in main memory

(Better CPU utilization in comparison with uni-programming OS)

OS

P1

P2

P3

Degree of Multiprogramming:

Number of running programs (Processes) in main memory (RAM)

As the degree of multiprogramming increases, CPU utilization also increases but upto a certain limit

Types of Multiprogramming:

- **Preemptive:** In pre-emptive scheduling, the operating system can interrupt a running process to allocate the CPU to another process usually due to priority rules or time-sharing policies.
- **Non-preemptive:** In non-preemptive scheduling, once a process starts using the CPU, it runs until it finishes or moves to a waiting state. The OS cannot forcibly take away the CPU.





Types of OS

- Multi-Tasking OS → Extension of multi-programming OS in which processes execute in round-robin fashion.
- Multi-User OS → This OS allows multiple users to access single system simultaneously
- Multi-Processing OS → This OS is used in computer systems with multiple CPUs
- Embedded OS → An OS for embedded computer systems.
- Real-Time OS → Real time operating systems (RTOS) are used in environments where a large number of events, mostly external to the computer system, must be accepted and processed in a short time or within certain deadlines.
- Hand-held Device OS → OS used in hand-held devices.
- e.g. Rocket Launching
- Shared Memory Distributed Memory





Stages of the Booting Process

Stage 1: Power-On:

- Electrical power flows to the motherboard and all connected components. PSU converts AC power to DC and distributes it to the CPU, RAM, hard drive, and other devices.
- This stage is purely hardware-based and happens in milliseconds.

Stage 2: POST (Power-On Self-Test)

- The CPU starts executing instructions from a special chip called the **BIOS (Basic Input/Output System)** or **UEFI (Unified Extensible Firmware Interface)** on modern systems.
- These are firmware programs stored in non-volatile memory on the motherboard.
- The first task is the **POST (Power-On Self-Test)**, POST checks if essential hardware components are functioning correctly e.g. CPU registers and functionality, RAM integrity and size, Keyboard and mouse presence, Video card and display, and Storage devices (hard drives, SSDs)
- If POST detects a problem, it typically emits a series of beeps to indicate the error.
- If everything checks out, the system proceeds to the next stage.

Stage 3: BIOS/UEFI Initialization

- After successful POST, the BIOS/UEFI initializes hardware devices and prepares them for use. This includes setting up communication channels between the CPU and other components.





Stages of the Booting Process

Stage 4: Boot Device Selection

- The BIOS/UEFI now needs to find the OS.
- It checks the boot device order. Common boot devices include: HDD, SSD, USB drive, and Network (for network booting)
- The system checks each device in order until it finds one with a valid boot sector.
- The boot sector is a special area on the storage device that contains instructions for loading the OS.
- If no bootable device is found, error is "No bootable device found" or "Insert boot disk and press any key."

Stage 5: MBR and Boot Loader Execution

- Once a bootable device is found, the BIOS reads the **Master Boot Record (MBR)** from the first sector of the disk (sector 0).
- The boot loader is a small program responsible for loading the OS kernel.
- Boot loader e.g. GRUB (Linux), NTLDR/BOOTMGR (Windows).





Stages of the Booting Process

Stage 6: Kernel Loading

- The boot loader locates the OS kernel on the disk and loads it into RAM. The kernel is the core of the OS, managing all hardware and software resources.
- At this point, control transfers from the BIOS/boot loader to the kernel. The kernel performs its own initialization:
 - Initializes device drivers
 - Sets up memory management structures
 - Initializes the process scheduler
 - Mounts the root file system

The kernel operates in privileged mode (kernel mode), giving it full access to all hardware resources. This is necessary for managing system resources and protecting the system from user programs.

Stage 7: Init Process and System Services

- After kernel initialization, the kernel starts the first user-space process, traditionally called **init** (process ID 1).
- On modern Linux systems, this might be systemd or another init system. On Windows, it's the Session Manager (smss.exe).

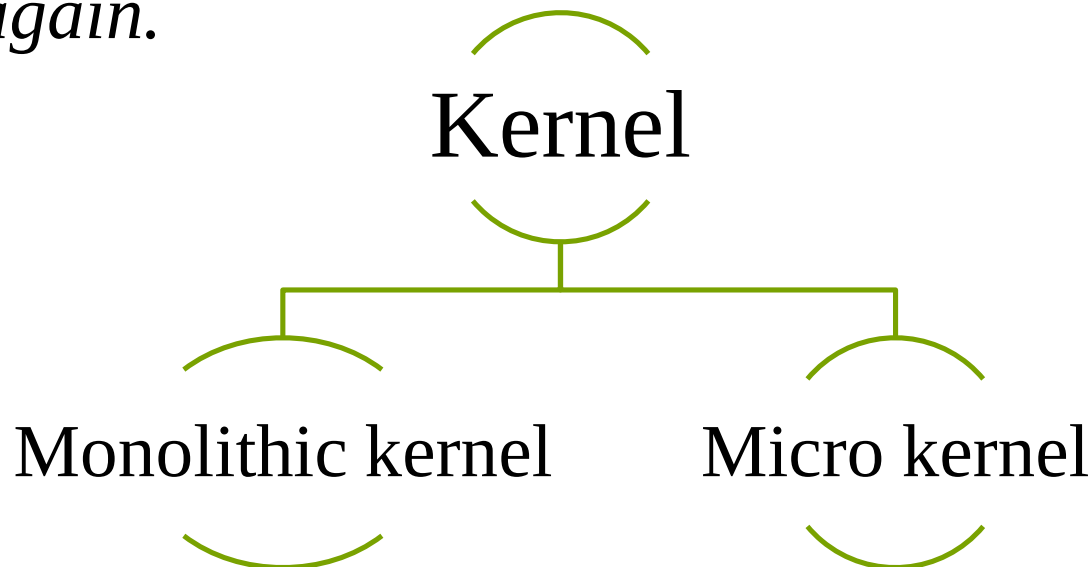
Stage 8: User Login and Desktop Environment





What is Kernel ?

- ❑ *The one program*
“running at all times on the computer”
is **the kernel**.
- ❑ Kernel **loads first** into memory when an OS is loaded and remains into memory until OS is shut down again.





Types of Kernel

Monolithic Kernel

- *In a monolithic kernel,*

***The same memory space is used to implement
user services and kernel services.***

- *The execution of processes is also faster than other kernel types as it does not use separate user and kernel space.*
- *If any service generates any error, it may crash down the whole system.*

Examples : Unix, Linux, Open VMS, XTS-400, etc.





Micro Kernel

- *In this, user services and kernel services are implemented into:*

*Two different address spaces:
user space and kernel space.*

- *Since it uses different spaces for both the services, so, the size of the microkernel is decreased, and which also reduces the size of the OS.*

Examples of Microkernel are L4, AmigaOS, Minix, K42, etc.





Dual-mode and Multimode of OS

- *The dual-mode operation provides a mechanism for enforcing **security, protecting system resources, and ensuring stability.***
- *There are two modes:*
 - *User Mode*
 - *Kernel Mode*
- *Let's understand each mode:*





User Mode

- *The processor executes user-level code, which includes applications and processes running on the system.*
- *User mode **restricts direct access to hardware***
- *User mode provides a limited set of instructions and **system calls** that allow user-level programs to request services from the kernel and perform specific tasks through well-defined interfaces.*





Kernel Mode

- *In kernel mode, the processor executes **kernel-level code**, which includes the core components of the OS, device drivers, and critical system services.*
- *Only code running in kernel mode can directly interact with hardware, access certain privileged instructions, and perform sensitive tasks that require elevated privileges.*





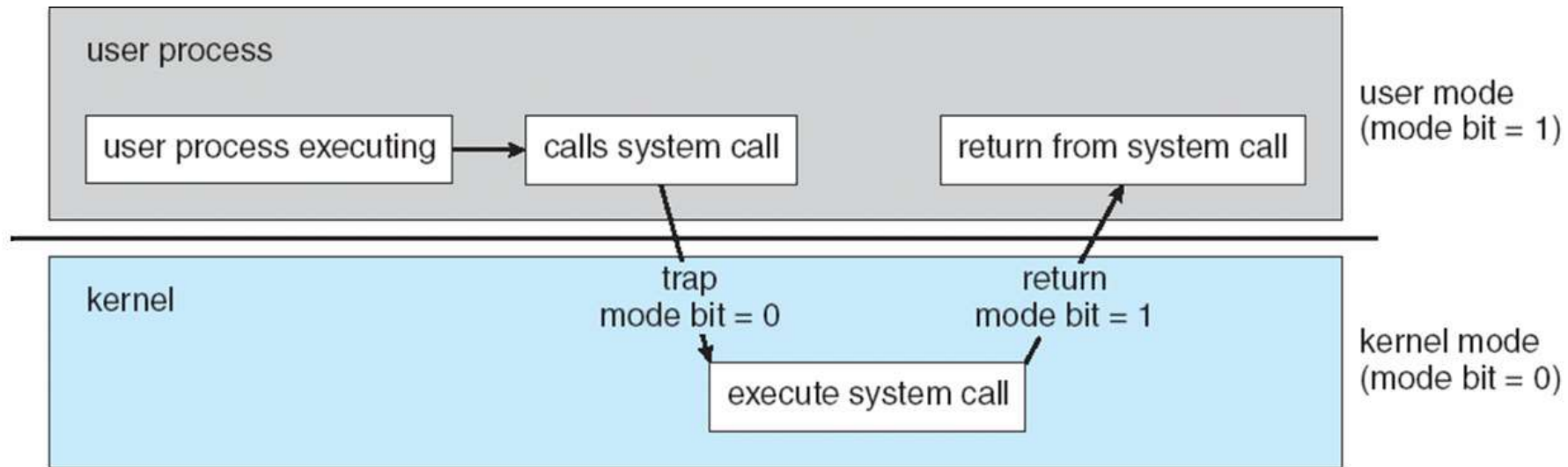
Dual-mode of OS

- A bit, called the **mode bit** is added to the hardware of the computer to indicate the current mode: **kernel (0) or user (1)**.
- With the mode bit, we are able to distinguish between a task that is executed on behalf of the operating system and one that is executed on behalf of the user.





Transition from User to Kernel Mode



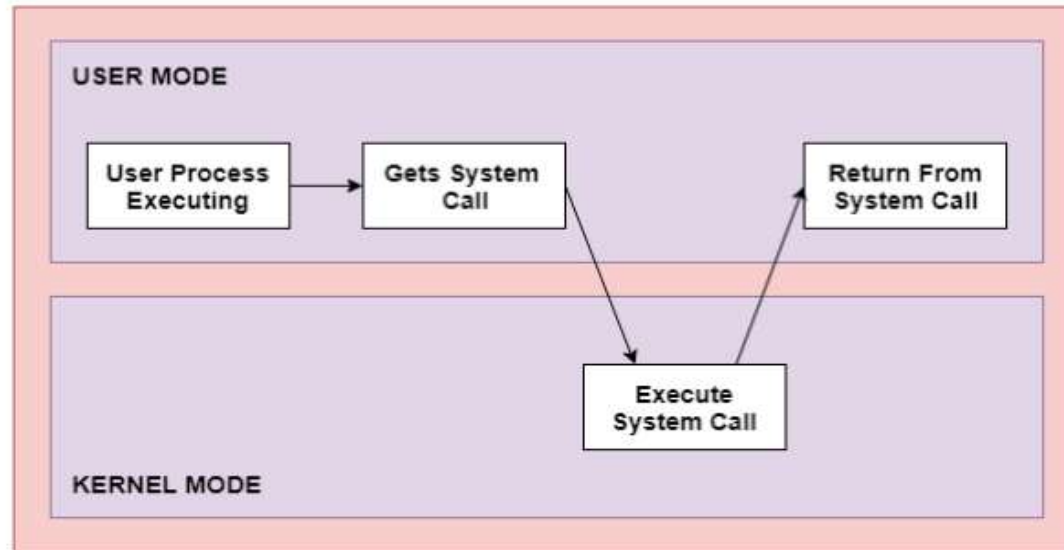
What is System call ?





System Call

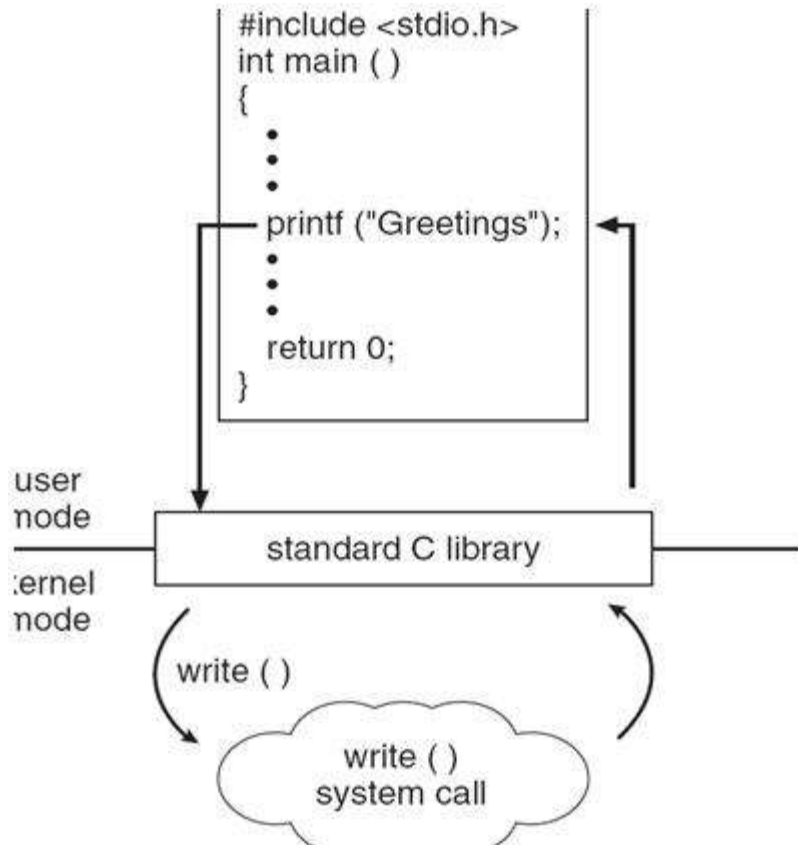
System call is the programmatic way in which computer program requests a services from the kernel of the operating system.





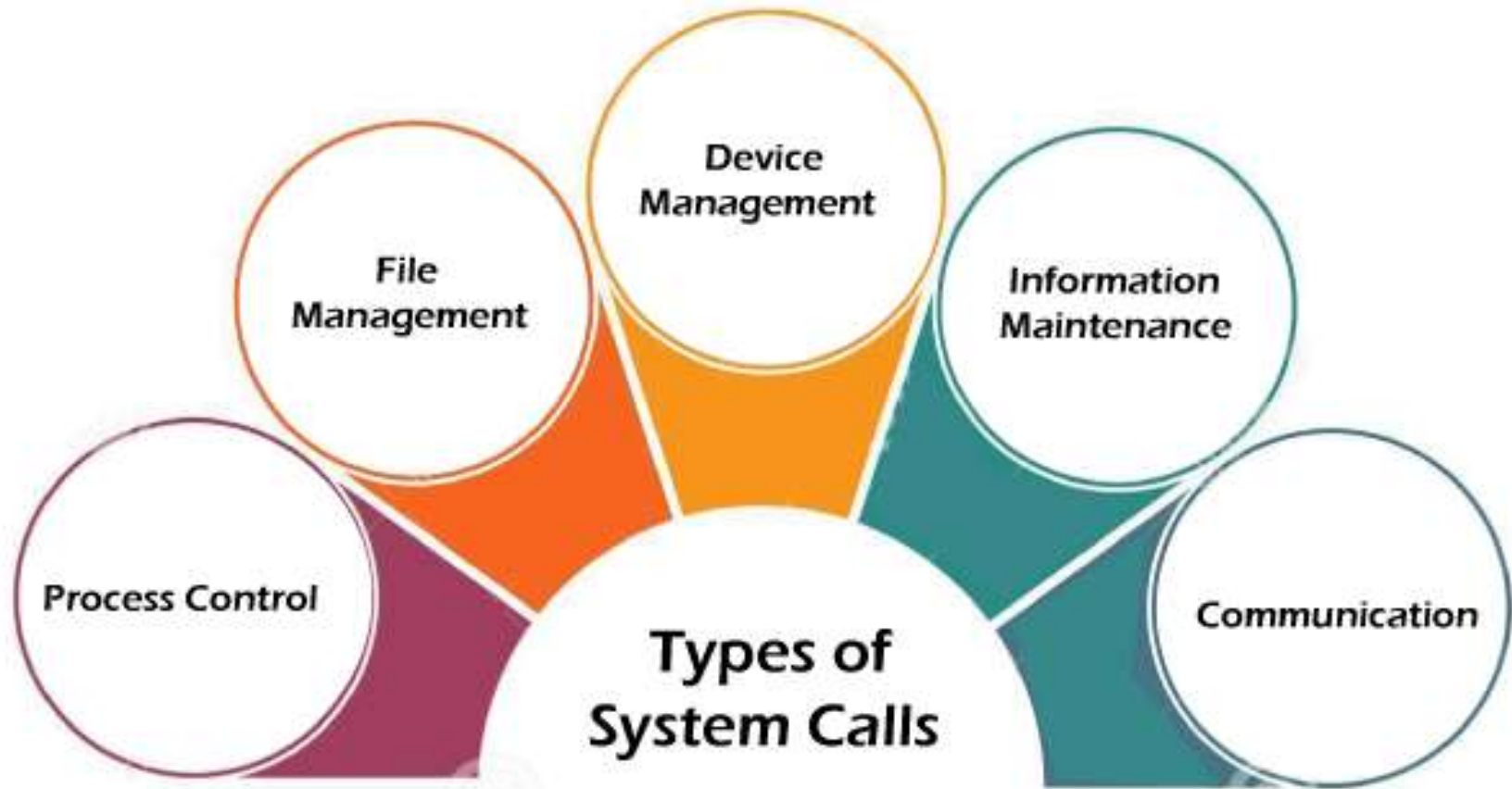
System Call

- The standard **C library** provides a portion of the system call interface for many version of UNIX and Linux.
- As an example, let's assume a C program invokes **printf()** statement.
- The C library intercepts this call and invokes the necessary system call(s) in the OS.
- The C library takes the value returned by **write()** and passes it back to the user program





System Call





System Call

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()





Thank You

